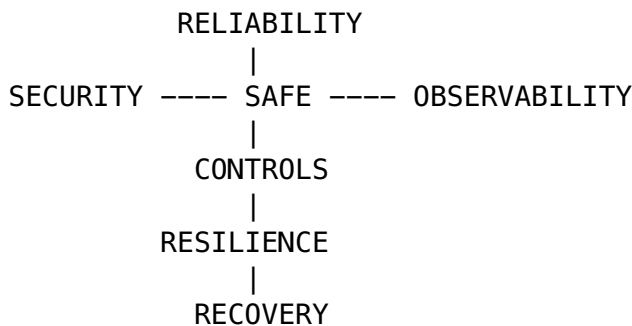


# DAY 5 — SRE, Observability, Security, Risk, Controls and Resilience

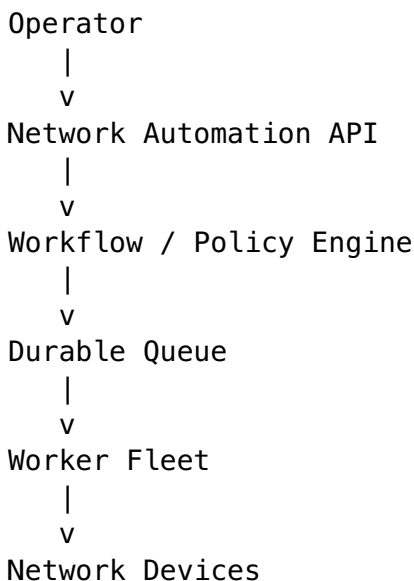
The central idea for today is:

**A production platform is not good because it works when everything is healthy. It is good because it behaves predictably when things fail, tells us what is happening, limits damage, protects critical assets, and can recover safely.**

For a critical network-automation platform, these concerns are inseparable:



We will use a recurring example:



Suppose the platform must safely modify routing configuration across 2,000 routers. Day 5 is about making sure that operation remains safe even when APIs, workers, databases, networks, credentials, devices, or entire regions fail.

## 1. Reliability vocabulary

These terms are related but mean different things.

| Concept      | Main question                          |
|--------------|----------------------------------------|
| Availability | Can I use the service now?             |
| Reliability  | Does it consistently behave correctly? |

| Concept         | Main question                                |
|-----------------|----------------------------------------------|
| Resilience      | Can it absorb and recover from failures?     |
| Durability      | Will my data survive?                        |
| Fault tolerance | Can it continue operating despite faults?    |
| Recoverability  | How effectively can it return after failure? |

## Availability

Availability means **the proportion of time a service is usable**.

If an automation API is available 99.9% of the time, approximately 0.1% downtime is permitted.

A service can be available but still unreliable.

Example:

---

API responds: HTTP 200  
But sends incorrect configurations to routers.

---

Technically the API is available.

Operationally it is extremely unreliable.

---

## Reliability

Reliability means:

| The system performs the expected function correctly and consistently over time.

For network automation:

---

Request:  
"Update NTP server on 500 routers"

Reliable behaviour:

- correct routers selected
- correct configuration generated
- change executed correctly
- failures detected
- result recorded
- no unrelated devices affected

---

Reliability includes much more than uptime.

---

## Resilience

Resilience is the ability to:

---

experience failure  
↓  
limit its impact  
↓  
continue where possible  
↓  
recover

---

Example:

---

Worker crashes while updating 500 routers.

Without resilience:  
Entire workflow fails.

With resilience:  
Queue notices unfinished work.  
Another worker resumes it.  
Only affected devices are retried.

---

## Durability

Durability is primarily about **data surviving failures**.

Suppose an approved change request has been stored.

Even if:

---

API crashes  
worker crashes  
node reboots  
cluster restarts

---

the change request should still exist.

Databases, replication, WAL/journaling, persistent storage and backups contribute to durability.

---

## Fault tolerance

Fault tolerance means the system can **continue functioning despite a component failure**.

For example:

---

3 API replicas

API-1 healthy  
API-2 crashes  
API-3 healthy

---

Load balancing moves traffic to API-1 and API-3.

Users may never notice the failure.

That is stronger than simply being recoverable.

---

## Recoverability

Recoverability asks:

| After something fails, how effectively can we restore service and state?

Examples:

- restore a database from backup
- rebuild a Kubernetes cluster
- fail over to another region
- replay unfinished jobs
- roll back a bad network change

A system may not tolerate every failure, but it should still be recoverable.

---

## 2. Failure domains and blast radius

A **failure domain** is a boundary within which a failure can occur.

A **blast radius** describes how much of the system is affected.

Consider these levels.

---

Application

|

Process

|

Pod

|

Node

|

Cluster

|

Data Centre

|

Region

---

### Process failure

Example:

---

Python worker process crashes.

---

Impact:

---

one execution worker

---

Normally Kubernetes or a supervisor restarts it.

Very small blast radius.

---

## Pod failure

A pod may fail because of:

- application crash
- OOM kill
- bad configuration
- health-check failure

If multiple replicas exist:

---

Pod A X

Pod B ✓

Pod C ✓

---

service continues.

---

## Node failure

The physical or virtual machine running many pods disappears.

Impact may include:

---

API pod

worker pods

telemetry agent

---

Kubernetes reschedules workloads to another node.

The blast radius is larger than a pod failure.

---

## Cluster failure

Possible causes:

- control-plane failure
- severe networking problem
- certificate failure
- broken cluster configuration
- operator mistake

Everything inside that cluster may become unavailable.

This is why extremely critical systems may use multiple clusters.

---

## Database failure

Possible impact:

---

API  
Workflow  
Inventory  
Audit  
Scheduler

---

A shared database can therefore create a very large failure domain.

---

## Queue failure

If the queue becomes unavailable:

---

API may accept requests  
    |  
    X  
workflow cannot dispatch jobs

---

If the queue loses messages, automation jobs may disappear.

Therefore queue durability matters.

---

## Network failure

Examples:

---

service ↔ database  
worker ↔ router  
cluster ↔ external authentication  
region A ↔ region B

---

Distributed systems must assume network communication can:

- fail
  - become slow
  - duplicate messages
  - partially succeed
- 

## Device failure

A single router might:

- reboot
- reject configuration
- be unreachable
- expose a broken API

Correct architecture prevents:

---

1 broken router  
    ↓  
entire 2,000-device workflow fails

---

## Data-centre failure

Power, cooling, network or infrastructure failures can make an entire site unavailable.

---

## Cloud-region failure

A region may lose:

- compute
- storage
- managed databases
- networking
- identity dependencies

Multi-AZ architecture does not automatically protect against complete regional failure.

---

## External dependency failure

Examples:

---

Identity provider  
DNS  
certificate service  
CMDB  
IPAM  
vendor API  
ticketing platform

---

Even if your platform is healthy, a critical dependency can make it unusable.

---

## Blast-radius reduction

Good architecture attempts to turn:

---

small failure  
↓  
small impact

---

rather than:

---

small failure  
↓  
system-wide outage

---

Techniques include:

- multiple replicas
- partitioning
- bulkheads
- bounded concurrency
- progressive deployment

- regional isolation
  - separate queues
  - per-device failure handling
  - rate limits
  - independent failure domains
- 

### 3. SRE fundamentals

SRE introduces a disciplined way to define reliability.

The important chain is:

---

SLI → SLO → Error Budget → Engineering Decision

---

#### SLI — Service Level Indicator

An SLI is **what you measure**.

Examples:

---

API availability  
API p95 latency  
automation success rate  
device configuration success rate  
job completion latency

---

For example:

---

Successful requests  
-----  
Total requests

---

could be an availability SLI.

---

#### SLO — Service Level Objective

An SLO is the **target** for an SLI.

Example:

---

SLI:  
Percentage of valid automation requests completed successfully.

SLO:  
99.9% successful over 30 days.

---

#### SLA — Service Level Agreement

An SLA is generally a **formal commitment**, often involving customers and consequences.

For example:



---

SLA:  
99.9% monthly API availability

If violated:  
service credits or contractual escalation

---

Therefore:

---

SLI = measurement  
SLO = engineering target  
SLA = contractual commitment

---

Usually the internal SLO should be stricter enough to protect the external SLA.

---

## Error budget

Suppose the SLO is:

---

99.9% success

---

Then:

---

$100\% - 99.9\% = 0.1\%$

---

is the permitted error budget.

If 100,000 automation jobs run:

---

Allowed failures  $\approx 100$  jobs

---

The idea is not:

| Fail 100 jobs deliberately.

It means engineering has explicitly defined how much unreliability the service can tolerate.

---

## Why error budgets are useful

Imagine a team wants to release rapidly.

Without an error budget:

---

Development:  
"We need faster releases."

Operations:  
"We need more stability."

---

These goals conflict indefinitely.

With an error budget:

---

Reliability healthy

↓

release normally

Error budget burning rapidly

↓

slow risky changes

focus on reliability

---

Reliability becomes measurable rather than emotional.

---

## Why not aim for 100%?

Because absolute availability usually requires disproportionate cost.

Attempting 100% may lead to:

- excessive redundancy
- fear of changing systems
- avoiding necessary maintenance
- huge operational cost
- extremely complicated architecture

Even upstream dependencies rarely provide true 100%.

The practical objective is:

| Reliability appropriate to the business need.

A DNS service may need a much higher SLO than an internal reporting dashboard.

---

## 4. Golden signals and service health

Google SRE popularised four particularly useful signals:

---

Latency

Traffic

Errors

Saturation

---

### Latency

How long does work take?

Examples:

---

API request: 120 ms

workflow creation: 450 ms

device operation: 8 sec

---

Prefer distributions such as:

---

p50  
p95  
p99

---

rather than relying only on averages.

---

## Traffic

How much demand is arriving?

Examples:

---

HTTP requests / second  
automation jobs / minute  
device commands / second  
queue messages / minute

---

## Errors

What percentage of operations fail?

Examples:

---

HTTP 5xx rate  
automation failure rate  
device command failure rate  
authentication failures

---

## Saturation

How close is the system to its limits?

Examples:

---

CPU 95%  
worker pool 100% busy  
DB connection pool exhausted  
queue continually growing

---

A service may still appear healthy while saturation is increasing.

That is often an early warning.

---

## RED

RED is especially useful for request-oriented services.

---

R = Rate  
E = Errors  
D = Duration

---

For an API:

---

How many requests?  
How many fail?  
How long do they take?

---

Very useful for microservices.

---

## USE

USE is especially useful for infrastructure resources.

---

U = Utilisation  
S = Saturation  
E = Errors

---

For a database:

---

CPU utilisation  
connection saturation  
disk errors

---

For a worker pool:

---

worker utilisation  
queued work  
worker failures

---

Think:

---

RED → services  
USE → resources  
Golden signals → broad service-health model

---

---

## 5. Monitoring vs observability

These are related but not identical.

### Monitoring

Monitoring answers questions you already anticipated.

Example:

---

Is CPU > 90%?  
Is API error rate > 5%?  
Is queue depth > 10,000?

---

### Observability

Observability means having enough information to understand **unexpected internal behaviour from external signals**.

Example:

An automation request took 47 seconds.

You did not already know why.

With good observability you can determine:

---

|            |        |
|------------|--------|
| API        | 100 ms |
| Policy     | 150 ms |
| Queue      | 22 s   |
| Worker     | 500 ms |
| Device API | 24 s   |

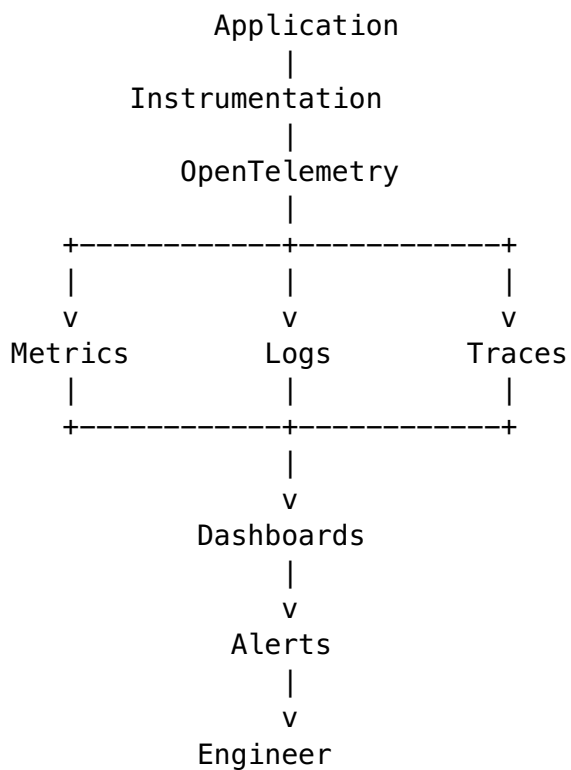
---

Now the cause becomes visible.

---

## Three telemetry pillars

---



---

### Logs

Tell us:

| What happened?

---

### Metrics

Tell us:

| How much, how often and how healthy?

---

## Traces

Tell us:

| Where did time and work go across distributed components?

---

## 6. Structured logging

Bad logging:

Something failed

---

Slightly better:

Router update failed

---

Structured logging:

```
{
  "timestamp": "2026-08-08T11:20:32Z",
  "severity": "ERROR",
  "service": "device-worker",
  "workflow_id": "wf-8421",
  "job_id": "job-9127",
  "device_id": "router-217",
  "trace_id": "abc123",
  "operation": "push_config",
  "error_type": "timeout",
  "duration_ms": 5000
}
```

---

Now logs can be searched and aggregated.

---

## Important fields

### Timestamp

When did the event occur?

Distributed systems should use consistent time standards.

---

### Severity

Typical levels:

DEBUG  
INFO  
WARN  
ERROR  
CRITICAL

---

Do not mark routine events as errors.

Otherwise operators learn to ignore logs.

---

## Correlation ID

Groups related operations.

For example:

---

```
User request
  |
correlation_id = CHG-9281
  |
API → workflow → worker
```

---

## Trace ID

Identifies a distributed trace.

---

## Workflow/job ID

Extremely useful in asynchronous systems.

A user request may finish quickly while the actual job continues for minutes.

---

## Device ID

Lets operators ask:

---

Show every operation involving router-217.

---

## Context propagation

A request begins with:

---

```
trace_id = 123
workflow_id = 456
```

---

These values should be propagated through:

---

```
API
  ↓
workflow
  ↓
queue message
  ↓
worker
  ↓
device adapter
```

---

Without propagation, logs become disconnected islands.

---

## Redaction

Never casually log:

- passwords
- API keys
- access tokens
- private keys
- complete secrets
- sensitive customer information

Logging systems frequently have wide access and long retention.

Therefore observability itself must be secured.

---

## 7. Metrics

Metrics represent numerical system behaviour.

### Counter

Only moves upward.

Examples:

---

```
requests_total  
automation_jobs_total  
device_failures_total
```

---

The process may reset counters when restarted.

---

### Gauge

Can increase or decrease.

Examples:

---

```
queue_depth  
active_workers  
database_connections  
devices_with_drift
```

---

### Histogram

Records observations in buckets.

Excellent for latency.

Example:



---

10 ms  
50 ms  
100 ms  
500 ms  
1 sec  
5 sec

---

From distributions we can estimate percentiles.

---

## Rate

Measures change over time.

Example:

---

```
requests_total = counter
```

```
rate(requests_total) =  
requests per second
```

---

## Percentile

A p95 latency of:

---

700 ms

---

means approximately 95% of requests completed within 700 ms.

The slowest 5% took longer.

This can reveal user pain hidden by averages.

---

## Useful network-automation metrics

---

```
api_requests_total  
api_request_duration  
automation_jobs_total  
automation_success_total
```

```
queue_depth  
queue_wait_duration
```

```
workers_active  
worker_utilisation
```

```
device_operations_total  
device_operation_failures
```

```
rollback_total  
drift_detected_total
```

---

You might derive:

---

Automation success rate  
=  
successful jobs / total jobs

---

or:

---

Device failure rate  
=  
failed device operations / total device operations

---

## 8. Distributed tracing

A **trace** represents an end-to-end distributed operation.

A **span** represents one operation inside that trace.

Example:

---

Trace: CHG-123

|                      |        |
|----------------------|--------|
| └─ API request       | 28 sec |
| └─ Authorisation     | 40 ms  |
| └─ Workflow creation | 100 ms |
| └─ Queue wait        | 2 sec  |
| └─ Device execution  | 25 sec |
| └─ Connect           | 1 sec  |
| └─ Push config       | 20 sec |
| └─ Verify            | 4 sec  |

---

## Parent and child spans

Suppose:

---

API request

---

creates:

---

workflow

---

The workflow span becomes a child of the API span.

This preserves causality.

---

## Trace context

The same trace ID must travel across service boundaries.

Especially:

---

```
API
|
v
Workflow
|
v
Queue
|
v
Worker
|
v
Device
```

---

For asynchronous queues, trace information often travels in message metadata.

Example:

---

```
Message:
  job_id
  device_id
  trace_id
  parent_span_id
```

---

---

## 9. Observability ecosystem

Think of these products by **role**, not installation details.

### Prometheus

Primarily:

---

Metrics collection + time-series storage + querying

---

Services expose metrics such as:

---

```
/api_requests_total
/queue_depth
```

---

Prometheus collects them.

---

### Grafana

Primarily:

---

```
Visualisation
Dashboards
Exploration
Alert views
```

---

For example:

---

## Network Automation Dashboard

|                 |        |
|-----------------|--------|
| API success     | 99.98% |
| p95 latency     | 310 ms |
| queue depth     | 1,240  |
| workers busy    | 78%    |
| device failures | 0.4%   |

---

## Elasticsearch

Stores and searches indexed data, commonly logs.

---

## Logstash

Processes and transforms logs.

Conceptually:

---

```
Raw log
  ↓
parse
  ↓
enrich
  ↓
transform
  ↓
Elasticsearch
```

---

## Kibana

Explores and visualises data stored in Elasticsearch.

Commonly used for log analysis.

---

## OpenTelemetry

OpenTelemetry provides vendor-neutral standards and tooling for instrumentation.

It helps applications emit:

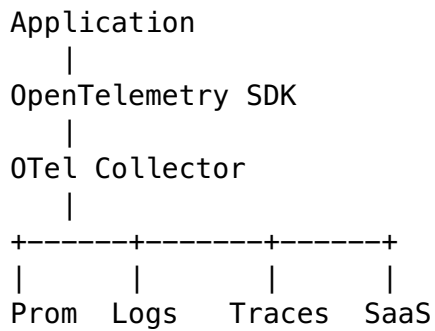
---

```
metrics
traces
logs
```

---

without tightly coupling application code to one observability vendor.

Conceptually:



## 10. Alerting

An alert should ideally mean:

| A human may need to act.

Not:

| Some number changed.

---

### Symptom-based alerts

Prefer alerting on user-visible problems.

Better:

Automation success rate < SL0

---

than:

worker CPU > 70%

---

High CPU may be perfectly healthy.

---

### Threshold alerts

Useful for clear limits.

Example:

disk utilisation > 90%

---

But thresholds should reflect operational meaning.

---

### Error-rate alerts

Example:

Device failure rate > 10%  
for 5 minutes

---

---

## Saturation alerts

Examples:

---

DB pool > 95%  
worker pool exhausted  
queue increasing continuously

---

## Alert deduplication

Imagine:

---

Database fails.

---

Then:

---

API alert  
workflow alert  
worker alert  
scheduler alert

---

You do not want 50 independent pages for one root incident.

Alert grouping and deduplication reduce noise.

---

## Severity

Example:

---

INFO  
WARNING  
SEV3  
SEV2  
SEV1

---

A SEV1 might mean major production impact requiring immediate response.

---

## Alert fatigue

If engineers receive:

---

100 alerts/day

---

but only two require action, they eventually ignore alerts.

A good alert should answer:

---

What is wrong?  
How serious is it?  
Who owns it?  
What should I investigate?

---

## SLO-based alerting

A sophisticated approach is to alert when the error budget is being consumed too rapidly.  
This focuses attention on actual reliability risk rather than arbitrary numbers.

---

## 11. Incident lifecycle

A useful model is:

---

Detect  
↓  
Triage  
↓  
Contain  
↓  
Mitigate  
↓  
Recover  
↓  
Analyse  
↓  
Improve

---

### Detect

Observability notices abnormal behaviour.

---

### Triage

Determine:

---

what is affected?  
how severe?  
when did it start?  
what recently changed?

---

### Contain

Stop the blast radius increasing.

Examples:

---

pause automation  
disable bad workflow  
isolate a region  
stop deployment  
open circuit breaker

---

## Mitigate

Reduce immediate impact.

Example:

---

route requests to healthy cluster

---

## Recover

Restore normal service.

---

## MTTR

Usually:

| Mean Time To Restore/Recover.

It measures how long service disruption typically lasts.

Reducing MTTR often requires:

- good observability
  - automation
  - runbooks
  - safe rollback
  - clear ownership
- 

## Root-cause analysis

RCA asks:

| Why was this failure possible?

Do not stop at:

---

"Engineer made mistake."

---

Ask deeper:



---

Why could one mistake affect 2,000 routers?

Why was there no validation?

Why was progressive rollout absent?

Why was rollback not automatic?

---

Strong engineering investigates the system that allowed the error.

---

## Post-incident review

A useful review captures:

- timeline
  - impact
  - detection
  - contributing factors
  - root causes
  - successful responses
  - unsuccessful responses
  - corrective actions
- 

## Corrective actions

A postmortem without improvements provides little value.

Actions might include:

---

add pre-change validation  
add canary rollout  
improve alert  
add test  
improve rollback  
reduce permission  
add capacity

---

Each important action needs ownership.

---

## 12. Resilience patterns

These patterns are fundamental to distributed systems.

### Timeout

Never assume a dependency will answer eventually.

---

Worker → Device

---

Without timeout:

---

Worker waits forever.

---

With timeout:

---

wait 10 seconds  
then fail safely

---

Timeouts prevent resource exhaustion.

---

## Retry

Some failures are temporary.

Example:

---

network timeout

---

Retrying may succeed.

But retries should generally be used only when operations are safely repeatable or protected by idempotency.

---

## Exponential backoff

Instead of:

---

retry every 100 ms

---

use approximately:

---

1 sec  
2 sec  
4 sec  
8 sec  
16 sec

---

This reduces pressure on failing dependencies.

---

## Jitter

If 10,000 workers all retry after exactly eight seconds:

---

dependency recovers

↓

10,000 requests arrive simultaneously

↓

dependency fails again

---

Jitter randomises retry timing.

For example:

---

7.1 sec  
8.6 sec  
6.9 sec  
9.2 sec

---

## How retries amplify outages

Consider:

---

API retries workflow 3 times  
Workflow retries worker 3 times  
Worker retries device 3 times

---

One user request can potentially trigger many downstream attempts.

Under failure:

---

dependency overloaded  
↓  
requests timeout  
↓  
clients retry  
↓  
even more requests  
↓  
dependency becomes more overloaded

---

This is a **retry storm**.

Retries therefore need:

- limits
  - backoff
  - jitter
  - time budgets
  - idempotency
  - clear retry ownership
- 

## Circuit breaker

If a dependency repeatedly fails:

---

Normal:  
requests allowed

Failures exceed threshold:  
circuit OPEN

Requests fail fast.

After recovery period:  
limited probes

Healthy:  
circuit CLOSED

---

This protects both systems.

---

## Bulkhead

Named after ship compartments.

Instead of:

---

all workloads share all workers

---

isolate capacity:

---

|                  |                 |
|------------------|-----------------|
| Critical changes | → worker pool A |
| Discovery        | → worker pool B |
| Reporting        | → worker pool C |

---

If reporting overloads its pool, critical operations remain available.

---

## Rate limiter

Controls how much work enters.

Example:

---

maximum 100 device changes/sec

---

Useful for:

- overload protection
  - fairness
  - external API limits
  - device protection
- 

## Backpressure

Backpressure means downstream saturation becomes visible upstream.

Without it:

---

Input 10,000 jobs/sec  
Processing 1,000 jobs/sec

Queue:

10K

20K

100K

1M

---

Eventually the system collapses.

Backpressure may:

- reject work
  - delay producers
  - reduce concurrency
  - expose "try later"
- 

## Graceful degradation

Instead of total failure, provide reduced functionality.

Example:

---

CMDB temporarily unavailable.

Instead of:

entire platform unavailable

Possibly:

read cached inventory

disable changes requiring fresh CMDB validation

allow read-only functions

---

The degraded mode must still be safe.

---

## 13. Capacity and saturation

Reliability problems are often capacity problems.

### Resource headroom

If normal utilisation is:

---

CPU 98%

---

there is almost no room for:

- traffic spikes
- retries

- node failure
- background work

A healthier architecture maintains headroom.

---

## Queue growth

Queue depth alone is not always alarming.

More important is the direction:

---

|            |                |
|------------|----------------|
| Incoming:  | 1,000 jobs/min |
| Processed: | 800 jobs/min   |

---

Every minute:

---

+200 backlog

---

The service is unsustainable.

---

## Connection limits

Databases, APIs and devices often have finite connection capacity.

Example:

---

DB maximum connections = 500

---

If every pod opens 50 connections and Kubernetes scales to 20 pods:

---

$20 \times 50 = 1000$  desired connections

---

The database becomes the bottleneck.

---

## Worker saturation

If every worker is busy:

---

new work waits  
queue increases  
latency increases  
timeouts occur  
retries occur  
load increases further

---

Saturation can cascade into outage.

---

## Database saturation

Potential causes:

- CPU exhaustion
- disk I/O
- excessive connections
- locks
- slow queries
- hot partitions
- transaction contention

A perfectly healthy API fleet cannot compensate for an overloaded database.

---

## 14. Failure testing

Waiting for production to reveal failure behaviour is a poor strategy.

We deliberately test failures.

### Fault injection

Inject controlled problems.

Examples:

---

```
kill pod
add network latency
drop packets
exhaust connection pool
make device API return 500
```

---

## Chaos engineering

Chaos engineering is more disciplined than randomly breaking systems.

Typical approach:

---

Hypothesis:

"If one worker node fails, automation continues."

Inject:

Terminate node.

Observe:

Do jobs resume?  
Is SLO violated?  
Is data lost?

---

## Dependency failure simulation

Test:

---

What if:  
database unavailable?  
queue slow?  
DNS broken?  
identity provider slow?  
CMDB responds 500?

---

## Recovery validation

Testing failure is only half the problem.

Also verify:

---

Can the system recover?

Does replication work?  
Does replay work?  
Can backups restore?  
Does rollback succeed?

---

## Why controlled failure improves confidence

Architecture diagrams describe what we **think** happens.

Failure tests reveal what **actually** happens.

---

## 15. Security principles

The classical foundation is the CIA triad.

---

Confidentiality  
Integrity  
Availability

---

### Confidentiality

Only authorised entities can see information.

---

### Integrity

Information and actions cannot be improperly changed.

For network automation, configuration integrity is extremely important.

---

### Availability

Authorised users can access the system when required.



Security attacks such as DDoS directly affect availability.

---

## Authentication

| Who are you?

Example:

Alice proves identity through OIDC.

---

## Authorization

| What are you allowed to do?

Alice may be authenticated but not authorised to modify production routers.

---

## Audit/accountability

| Who performed the action, and can we prove it?

---

## Least privilege

Give an identity only the permissions it requires.

Bad:

worker = global administrator

---

Better:

inventory-worker = read inventory  
config-worker = update approved device groups

---

## Defence in depth

Do not rely on one security barrier.

---

Authentication

↓

Authorization

↓

Network controls

↓

Application validation

↓

Device authorization

↓

Audit

---

If one control fails, others remain.

---

## Zero trust

Zero trust does not literally mean "trust nothing."

It means:

| Do not grant trust merely because something is inside the corporate network.

Continuously verify:

- identity
  - authorization
  - context
  - workload
  - requested action
- 

## 16. Threat modelling

Threat modelling asks:

| What are we protecting, from whom, and how could it be attacked?

### Asset

Something valuable.

Examples:

---

production network  
router credentials  
configuration repository  
audit records  
customer traffic

---

### Threat

Something capable of causing harm.

Example:

---

unauthorised configuration modification

---

### Attack surface

Every point through which the system can be attacked.

For example:

---

Public API  
Admin UI  
CI/CD  
Device APIs  
Queue  
Database  
Secrets system

---

## Trust boundary

A place where trust level changes.

Example:

---

Internet  
|  
--- trust boundary ---  
|  
API Gateway

---

or:

---

Automation platform  
|  
--- trust boundary ---  
|  
Network device

---

## Threat actor

Could be:

- external attacker
  - compromised user
  - compromised service
  - malicious insider
  - compromised dependency
- 

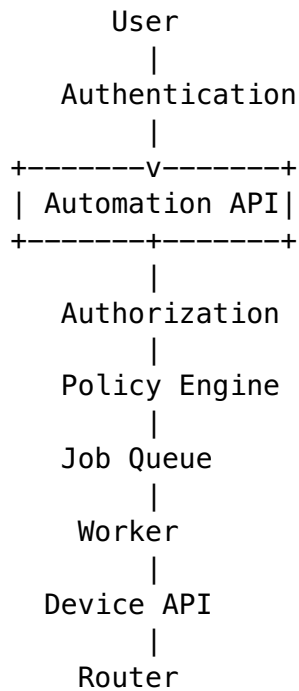
## Mitigation

Control that reduces threat likelihood or impact.

---

## Example threat model

---



### Possible threats:

stolen access token  
unauthorised production change  
job-message tampering  
worker credential theft  
device impersonation  
malicious configuration payload  
audit-log modification

---

### Mitigations:

short-lived tokens  
RBAC  
mTLS  
signed artifacts  
input validation  
least privilege  
immutable audit  
network segmentation

---

## STRIDE

A useful high-level framework:

---

S – Spoofing  
T – Tampering  
R – Repudiation  
I – Information disclosure  
D – Denial of service  
E – Elevation of privilege

---

You do not need to force every architecture through STRIDE, but it provides a systematic checklist.

---

## 17. OAuth2, OIDC and tokens

This distinction is important.

### OAuth2

Primarily:

| Authorisation delegation.

It answers questions like:

---

Can application A access resource B  
with these permissions?

---

### OIDC

OpenID Connect adds an identity/authentication layer on top of OAuth2 concepts.

It answers:

---

Who is this user?

---

Simplified:

---

OAuth2 → access delegation  
OIDC → identity/authentication

---

### Access token

A credential presented to an API.

Example:

---

Authorization: Bearer <token>

---

The API validates the token and its permissions.

Access tokens should generally be short-lived.

---

## Scope

Represents permitted operations.

Example:

---

```
inventory:read  
changes:create  
changes:approve  
devices:write
```

---

## Refresh token

Allows a client to obtain another access token without repeating the entire login flow.

Refresh tokens usually live longer and therefore require stronger protection.

Machine-to-machine systems often use mechanisms such as workload identity instead.

---

## JWT

JWT is a token format.

Conceptually:

---

```
Header  
Payload  
Signature
```

---

Payload may contain claims:

---

```
subject  
issuer  
audience  
expiry  
roles/scopes
```

---

Important:

| JWT is not synonymous with OAuth2.

OAuth2 can use different token formats.

---

## 18. PKI, certificates and mTLS

### Public/private keys

A key pair contains:

---

```
Private key → secret  
Public key  → can be shared
```

---

The private key should never be distributed casually.

---

## Certificate

A certificate associates an identity with a public key.

Conceptually:

---

Identity  
Public key  
Validity period  
Issuer  
Signature

---

## Certificate authority

A CA signs certificates.

If you trust the CA, you can trust certificates that validate through the expected chain and policy.

---

## Trust chain

---

Root CA  
|  
Intermediate CA  
|  
Service Certificate

---

## TLS

TLS provides encrypted network communication and normally authenticates the server.

For example:

---

Worker  
|  
HTTPS/TLS  
|  
Device API

---

## Mutual TLS

Normal TLS:

---

client verifies server

---

mTLS:

---

client verifies server  
AND  
server verifies client

---

Useful for strong service-to-service identity.

---

Workflow Service  
|  
mTLS  
|  
Device Gateway

---

---

## Certificate rotation

Certificates expire or may need revocation.

Therefore systems need:

---

issue  
deploy  
monitor expiry  
rotate  
revoke

---

Manual certificate management does not scale well across thousands of workloads.

---

## 19. Identity for machines

Humans and machines should generally use different identity mechanisms.

### Service accounts

A workload-specific identity.

Example:

---

device-worker-service-account

---

### Workload identity

The platform provides an identity to the running workload.

Instead of:

---

PASSWORD=supersecret

---

stored permanently inside configuration, the workload can obtain short-lived credentials based on its verified identity.

---



## Short-lived credentials

Preferred because a stolen credential automatically becomes useless after expiry.

Compare:

---

API key valid for 5 years

---

versus:

---

token valid for 15 minutes

---

## API keys

Simple but often long-lived.

Risks:

- copying
- accidental logging
- source-control leakage
- difficult attribution
- difficult rotation

Use stronger identity systems where possible.

---

## Certificates

Can provide strong machine identity, especially for mTLS.

---

## Why not human credentials?

Never build:

---

```
worker
|
uses Alice's username/password
|
router
```

---

Problems include:

- password rotation
- employee departure
- poor auditability
- excessive permissions
- impossible ownership
- MFA conflicts

Machine identities should represent machines.

---

## 20. Secure coding

Critical infrastructure services should assume incoming data may be hostile.

### Input validation

Validate:

- type
- format
- allowed values
- size
- range
- relationships

For device commands, prefer strict schemas and allowlists.

---

### SQL injection

Dangerous:

---

```
"SELECT ... WHERE user='" + input + "'"
```

---

User-controlled input can alter the query.

Use parameterised queries.

---

### Command injection

Especially dangerous in automation systems.

Bad conceptual design:

---

```
shell("ping " + user_input)
```

---

Attacker may provide shell syntax.

Prefer:

- native libraries
- structured command arguments
- strict allowlists

Never construct shell commands from arbitrary user input.

---

### SSRF

Server-Side Request Forgery occurs when an attacker makes your server connect to unintended destinations.

Dangerous automation input:

---

```
"fetch config from this URL"
```

---

could potentially target internal services.

Mitigations include:

- destination allowlists
  - network egress controls
  - URL validation
  - blocking metadata/internal endpoints
- 

## Path traversal

Input such as:

---

```
../../secrets
```

---

may escape an intended directory.

Canonicalise and constrain file paths.

---

## XSS

Primarily browser-related.

Malicious data becomes executable JavaScript when rendered in a UI.

Backends still matter because they may store or return attacker-controlled content.

---

## CSRF

Tricks an authenticated browser into submitting unwanted requests.

Primarily relevant to browser/session-based applications.

---

## Secrets leakage

Common sources:

---

```
logs  
exception traces  
Git repositories  
Docker images  
CI output  
configuration files
```

---

---

## Dependency vulnerabilities

Your own code may be secure while a third-party library is vulnerable.

This leads directly to supply-chain security.

---

## 21. Kubernetes security

Kubernetes creates many powerful security boundaries.

### RBAC

Controls:

---

who  
can perform what action  
on which resource

---

Example:

---

worker:  
  read ConfigMaps  
  read specific Secrets  
  update Jobs

NOT:  
  cluster-admin

---

### Service accounts

Pods should have identities appropriate to their function.

Do not give every workload the same Kubernetes identity.

---

### NetworkPolicy

Controls pod-to-pod/network communication.

Instead of:

---

every pod → every pod

---

prefer:

---

API → workflow  
workflow → queue  
workers → devices

---

only where required.

---

## Pod security

Prefer:

- non-root containers
  - minimal Linux capabilities
  - no privileged mode unless necessary
  - read-only filesystem where practical
  - restricted host access
  - appropriate seccomp/security settings
- 

## Secrets

Kubernetes Secrets are useful for distributing sensitive configuration but must still be protected.

Consider:

- encryption at rest
  - RBAC
  - external secret managers
  - rotation
  - avoiding environment/log leakage
- 

## Admission control

Admission controllers evaluate Kubernetes requests before resources are accepted.

They can enforce policies such as:

---

No privileged containers.  
Only approved registries.  
Resource limits required.  
Signed images required.

---

## Image security

Use:

- minimal images
  - patched base images
  - scanning
  - trusted registries
  - immutable tags/digests
  - signatures/provenance
-

## Least privilege

The principle should apply at every level:

---

user  
service account  
pod  
container  
network access  
cloud IAM  
database  
device privileges

---

## 22. Software supply-chain security

Software entering production has a supply chain:

---

Developer  
↓  
Source code  
↓  
Dependencies  
↓  
Build  
↓  
Container image  
↓  
Registry  
↓  
Deployment

---

Every stage can be attacked.

---

## Dependency scanning

Detect vulnerable libraries.

---

## Image scanning

Detect vulnerabilities inside container images.

---

## SAST

Static Application Security Testing analyses source code or compiled artifacts without executing the application.

Useful for identifying certain unsafe patterns.

---

## DAST

Dynamic Application Security Testing tests a running application from the outside.

---

## SBOM

Software Bill of Materials.

Think:

---

"ingredient list for software"

---

It identifies components contained in an artifact.

When a new vulnerability appears, organisations can ask:

---

Which deployed systems contain library X?

---

## Artifact signing

Cryptographically proves an artifact was produced by an expected source.

---

## Provenance

Answers:

---

Who built this?

From which source commit?

Using which build process?

---

## Trusted registries

Production clusters should generally pull images only from approved repositories.

---

## Why CI/CD is a security boundary

A CI/CD system may be able to:

- access source code
- retrieve secrets
- create trusted artifacts
- publish containers
- deploy production
- modify infrastructure

Compromising CI/CD can therefore compromise the entire platform.

Treat pipeline permissions as production permissions.

---

## 23. Vulnerability management

Vulnerability management is a lifecycle.

---

Discover  
↓  
Assess severity  
↓  
Prioritise  
↓  
Remediate  
↓  
Verify  
↓  
Track exceptions

---

### Discover

Sources include:

- scanners
  - vendor advisories
  - penetration tests
  - dependency tooling
  - security research
- 

### Assess severity

Consider:

---

technical severity  
exploitability  
exposure  
business importance

---

A critical library vulnerability on an isolated development machine may represent less immediate risk than a medium vulnerability exposed directly to the internet.

---

### Prioritise

Risk-based prioritisation is better than blindly sorting CVSS scores.

---

### Remediate

Options:

- patch
- upgrade
- replace component
- change configuration
- isolate service



---

## Verify

Confirm the vulnerability is actually removed.

---

## Exceptions

Sometimes remediation cannot happen immediately.

Then explicitly record:

---

risk  
owner  
reason  
compensating controls  
expiry/review date

---

An exception should not mean "ignore forever."

---

## 24. Technology and operational risk

A useful definition:

Technology risk is the possibility that technology failure, misuse, vulnerability or uncontrolled change causes harm to the organisation.

Major categories include:

### Availability risk

Critical system unavailable.

---

### Cybersecurity risk

Attack compromises confidentiality, integrity or availability.

---

### Data risk

Data may be:

- lost
  - corrupted
  - exposed
  - inaccurate
  - improperly retained
- 

### Change risk

A deployment or configuration modification causes failure.

Network automation makes this particularly important because one automation job can affect thousands of devices.

---

## Operational risk

Problems caused by processes, people or systems.

Examples:

---

incorrect runbook  
manual mistake  
unclear ownership  
missing monitoring  
poor escalation

---

## Third-party risk

Dependence on:

- cloud providers
- SaaS
- vendors
- identity providers
- libraries
- managed databases

creates risk outside your direct control.

---

## 25. Risk assessment

The basic idea is:

---

Likelihood

x

Impact

↓

Risk

---

Do not become obsessed with exact mathematics.

The purpose is prioritisation.

---

## Inherent risk

Risk before considering controls.

Example:

---

Automation service has administrator access  
to every production router.

Inherent risk = very high

---

## Mitigation

Add controls:

---

RBAC  
approval  
progressive execution  
validation  
mTLS  
audit  
rollback

---

## Residual risk

Risk remaining after mitigations.

---

Inherent risk  
↓  
Controls  
↓  
Residual risk

---

Residual risk rarely becomes literally zero.

---

## 26. Controls

Controls help prevent, detect or correct undesirable events.

### Preventive controls

Stop the event.

Examples:

---

RBAC  
schema validation  
peer review  
policy checks  
approval  
maintenance-window restriction

---

Network example:

---

Configuration deleting all BGP neighbours  
is rejected before deployment.

---

### Detective controls

Discover an undesirable event.

Examples:

---

monitoring  
drift detection  
audit review  
security alerts  
configuration comparison

---

## Corrective controls

Help recover after an event.

Examples:

---

rollback  
restore from backup  
configuration recovery  
credential rotation  
failover

---

## Combined example

Change submitted

Preventive:  
policy blocks dangerous configuration.

Detective:  
post-change verification detects routing loss.

Corrective:  
automatic rollback restores previous configuration.

---

Strong systems generally need all three.

---

## 27. Control design and effectiveness

There is a major difference between:

---

"We have a control."

---

and:

---

"The control actually works."

---

Example policy:

| Production changes require approval.

But imagine everyone has permission to bypass the approval API.

The control exists on paper but is ineffective.

---

## Ownership

Every important control needs a responsible owner.

---

## Evidence

You should be able to prove the control executed.

Example:

---

change ID  
approver  
timestamp  
policy result  
deployment result

---

## Frequency

A control performed once per year may not adequately address a daily risk.

---

## Automation

Automated controls are often:

- more consistent
- easier to evidence
- faster
- harder to forget

Example:

---

Policy engine automatically validates  
every production change.

---

## Exceptions

Controls sometimes require exceptions.

Good exceptions are:

---

explicit  
approved  
time-bounded  
documented  
reviewable

---

## 28. Change governance

Network change is one of the largest operational risks.

A good flow may look like:

---

Change request

↓

Peer review

↓

Risk classification

↓

Approval

↓

Execution

↓

Validation

↓

Evidence

---

## Change request

Records:

---

what

why

where

when

expected result

rollback plan

---

## Peer review

Another engineer evaluates technical correctness.

---

## Risk classification

Not every change needs identical governance.

Example:

---

Low risk:

update description on one lab switch

Medium:

change 10 access switches

High:

modify BGP policy across production backbone

---

Governance should be proportional.

---

## Approval

Certain risk classes may require explicit approval.

---

## Segregation of duties

A person should not necessarily be able to:

---

write  
approve  
execute  
hide evidence

---

for the same critical change.

That concentration of power creates risk.

---

## Maintenance window

Some disruptive changes should occur only in approved periods.

---

## Emergency change

Sometimes urgent production problems require faster procedures.

Emergency does not mean:

---

no governance

---

It means an appropriately accelerated governance process followed by review.

---

## Validation

Before:

---

Is the proposed state safe?

---

After:

---

Did the intended result actually happen?

---

## Rollback

A change without a recovery strategy is risky.

---

## Evidence

The platform should automatically retain proof of what occurred.

---

## Can strong governance coexist with fast engineering?

Yes.

Bad governance:

---

ticket  
manual email  
spreadsheet  
three meetings  
human typing commands

---

Good engineering governance:

---

policy as code  
automated testing  
automated evidence  
risk-based approval  
progressive rollout  
automatic validation  
automatic rollback

---

The goal is:

| Make the safe path the fastest path.

---

## 29. Auditability

For a critical action we should be able to answer:

---

WHO?  
WHEN?  
WHY?  
WHAT WAS APPROVED?  
WHAT CHANGED?  
WHAT HAPPENED?

---

Therefore capture:

### Identity

Who requested, approved and executed?

---

### Timestamp

When did each event occur?

---

### Reason

Why was the change needed?

---



## Approval

Who authorised it?

---

## Old state

Previous BGP policy = v17

---

## New state

New BGP policy = v18

---

## Execution outcome

Device 1 success  
Device 2 success  
Device 3 rollback

---

## Evidence

Examples:

---

diff  
policy evaluation  
test result  
device response  
verification output

---

For highly critical environments, audit data may need protection against modification.

---

## 30. Data protection

### Data classification

Not all data has equal sensitivity.

Example:

---

Public  
Internal  
Confidential  
Restricted

---

Credentials should receive stronger protection than public documentation.

---

### Encryption in transit

Protect data travelling between systems.

Usually:

---

TLS

---

## Encryption at rest

Protect stored data.

Examples:

---

database encryption  
encrypted disks  
object-store encryption  
backups

---

## Key management

Encryption is only as trustworthy as its key management.

Keys need:

- controlled access
  - rotation
  - secure storage
  - auditing
- 

## Secret management

Prefer central secret-management systems over:

---

passwords in source code

---

Important capabilities include:

---

storage  
access policy  
rotation  
versioning  
auditing

---

## Retention

Do not keep everything forever.

Retention policies define:

---

what data  
for how long  
and why

---

## Redaction

Sensitive values should be masked before reaching logs and telemetry.

Example:

---

```
token=eyJh...
```

---

should not appear in full.

---

## 31. Business continuity and disaster recovery

These concepts are often confused.

### High availability

HA attempts to keep the service running through ordinary component failures.

Example:

---

```
Node A fails  
Node B continues
```

---

### Disaster recovery

DR addresses major failures where normal HA may be insufficient.

Example:

---

```
Entire cloud region unavailable.
```

---

### Business continuity

Broader question:

| Can the organisation continue its important business activities during disruption?

Technology DR is one component of business continuity.

---

### Backup

Backup creates a recoverable copy of data.

Important:

| Replication is not the same as backup.

If someone corrupts the primary database:

---

Primary DB corruption

↓

Replication

↓

Replica DB corruption

---

A separate backup may still allow recovery.

---

## Restore

A backup has limited value if you cannot restore it successfully.

Therefore:

---

Backup success

≠

Recovery success

---

Restore testing is essential.

---

## RTO — Recovery Time Objective

| How quickly must the service recover?

Example:

---

RT0 = 1 hour

---

means the business expects service recovery within approximately one hour after the applicable disaster scenario.

---

## RPO — Recovery Point Objective

| How much data can we afford to lose?

Example:

---

RP0 = 5 minutes

---

means recovery should lose no more than roughly five minutes of data according to the designed recovery strategy.

---

## RTO vs RPO

Easy mental model:

---

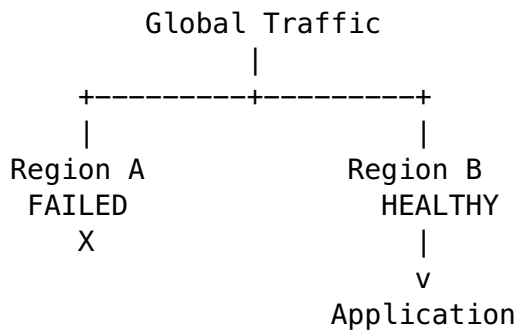
RT0 → TIME to recover

RP0 → DATA loss tolerated

---

## Regional failure

Imagine:



Recovery may involve:

- DNS/global load-balancer changes
  - database failover
  - queue recovery
  - workload startup
  - secret availability
  - traffic validation
- 

## Recovery procedures

DR should specify:

---

How do we detect disaster?  
Who declares DR?  
How do we fail over?  
How is data recovered?  
How do we validate?  
How do we communicate?  
How do we return to primary?

---

## Recovery testing

The first time you execute your DR plan should not be during the real disaster.

Regularly test:

---

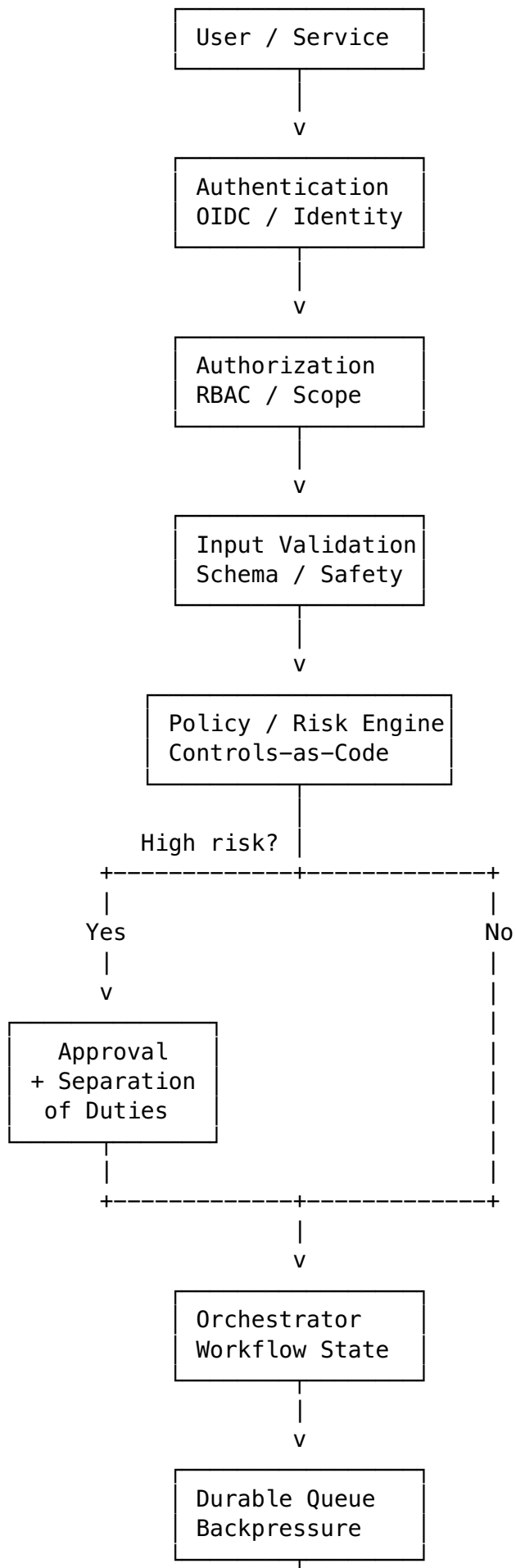
backup restore  
regional failover  
credential recovery  
database recovery  
queue replay  
cluster rebuilding

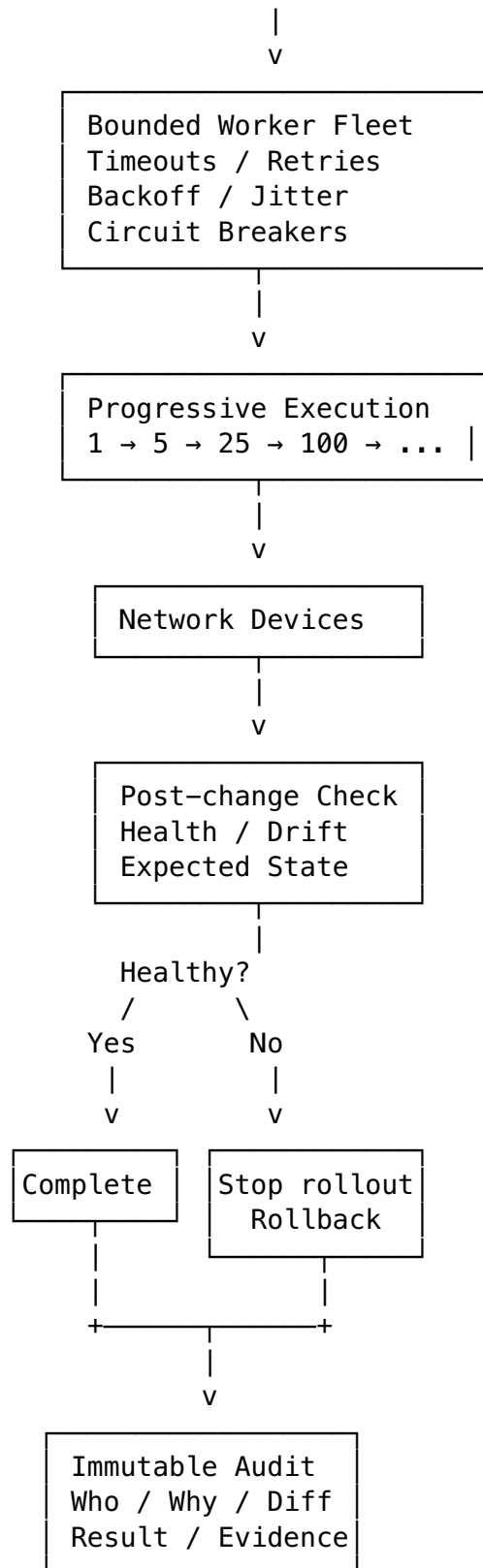
---

## **32. Integrated safe-change architecture**

Now combine everything.

Imagine an engineer requests a production routing change.

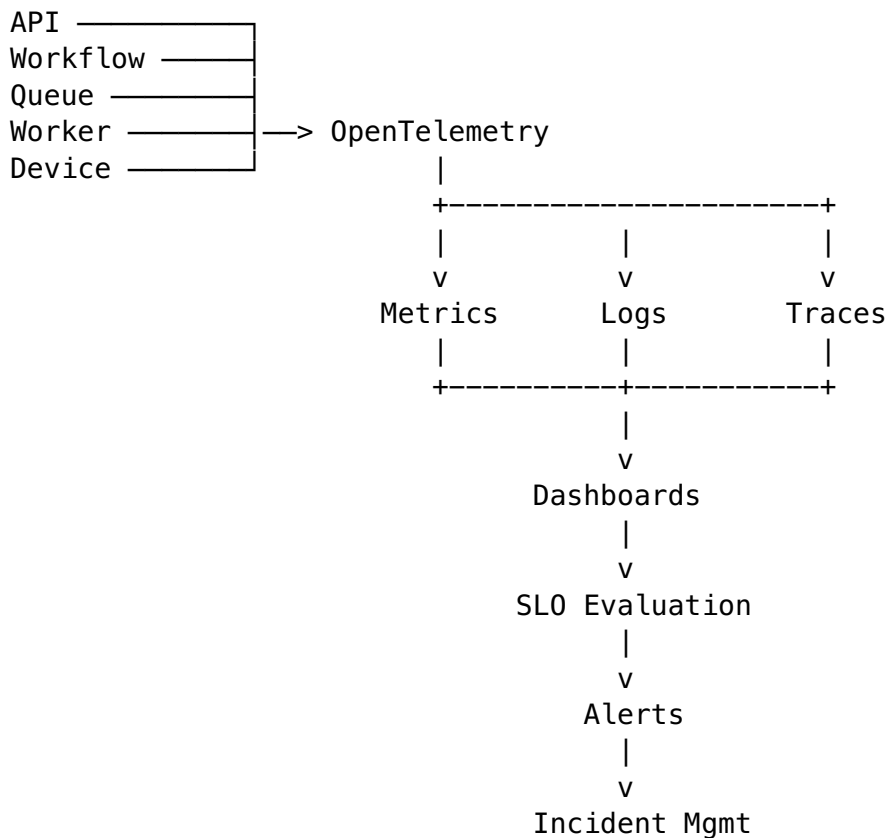




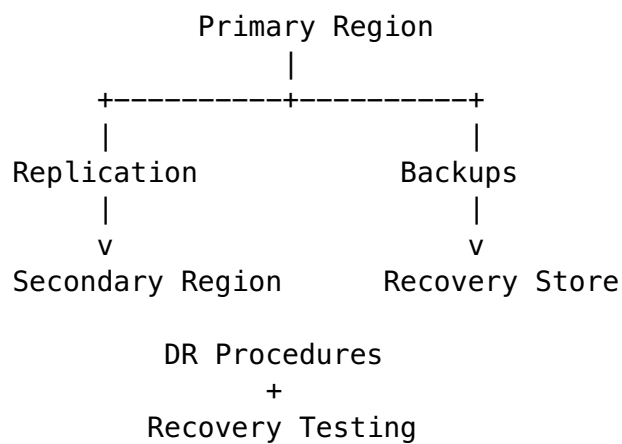
---

At the same time, every important component emits telemetry:





And underneath all of this:



## How all the disciplines fit together

Consider a dangerous routing change.

### Security asks

Who is requesting this?  
Are they allowed?  
Are credentials protected?  
Can the request be trusted?

---

## Controls ask

---

Was it reviewed?  
Was policy evaluated?  
Does it require approval?  
Can we prove that approval occurred?

---

## Reliability asks

---

Will the system perform the change correctly?

---

## Resilience asks

---

What if the queue, worker, network or router fails?

---

## Observability asks

---

Can we see exactly what is happening?

---

## SRE asks

---

Is service behaviour within its reliability objectives?

---

## Risk asks

---

What could go wrong?  
How likely is it?  
What would the impact be?  
Have we reduced the risk sufficiently?

---

## Audit asks

---

Can we reconstruct exactly what happened later?

---

## Disaster recovery asks

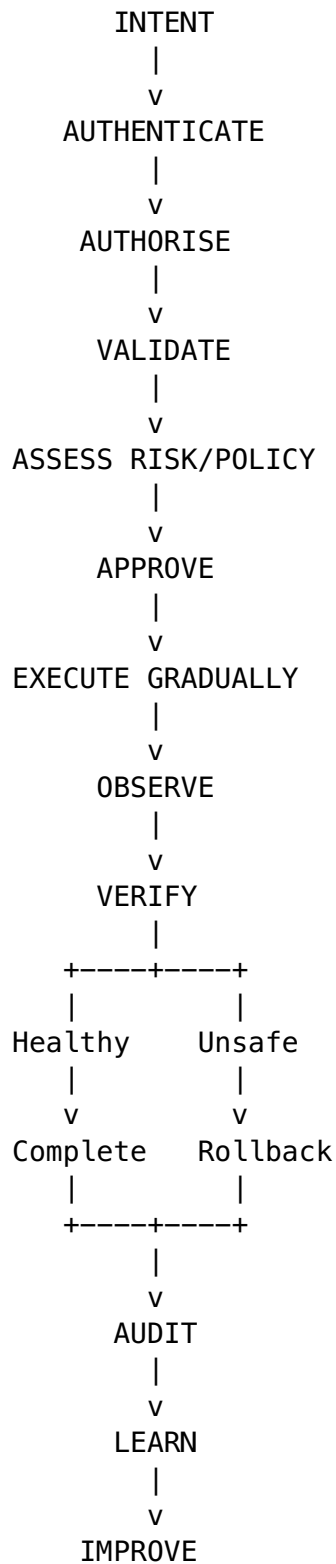
---

If the entire platform or region is lost,  
how do we restore service and state?

---

These are therefore not independent disciplines.

They form one loop:



The mature engineering mindset is therefore not:

---

Reliability team  
Security team  
Risk team  
Operations team  
Development team

---

working as separate worlds.

It is:

**Every production change is designed to be authenticated, authorised, validated, controlled, observable, bounded in blast radius, resilient to failure, verifiable, reversible, auditable and recoverable.**

That is the foundation of operating critical infrastructure safely at scale.